

MODULE - 1

SOLID principles

- S - Single Responsibility
- O - Open / close
- L - Liskov Substitution
- I - Interface Segregation
- D - Dependency Inversion.

SOLID is a Five Design principle that
Make Code easier to understand, Maintain
extend and test

SRP (Single Responsibility principle)

A class should have only one Reason to

Change

Eg: class Invoice {

void calculateTotal() { }

}

class InvoicePrinter {

void printerInvoice { }

}

OCP (Open/Close principle)

Software Entities should be open for extension but closed for Modification.

Eg:

```
Interface Shape {  
    double area();  
}  
  
class Circle implements Shape {  
    public double area()  
    {  
        return 3.14 * 5 * 5;  
    }  
}
```

Liskov Substitution principle (LSP)

A Subclass should be able to replace its parent class without affecting program behaviour.

Eg:

```
class Bird {  
    void fly() {}  
}  
  
class Sparrow extends Bird {}
```

I - Interface Segregation principle (ISP)

Clients should not be forced to implement interfaces they do not use.

Eg:

```
interface printer {  
    void print();  
}  
interface Scanner {  
    void scan();  
}
```

D - Dependency inversion principle (DIP)

High-level modules should depend on abstraction not concrete classes

Eg:

```
interface Keyboard {}
```

```
class WiredKeyboard implements Keyboard {}
```

```
class Computer {
```

```
    Keyboard keyboard;
```

```
    Computer (Keyboard keyboard)
```

```
    {  
        this.keyboard = keyboard;  
    }  
}
```